CASE STUDY

Demand Forecasting

Preserve forecasting continuity when product codes change. Build a REST API that links old and new codes into product families.

Open Live API → | GitHub Repository | See an Example

The Problem

Accurate **demand forecasting** drives production planning, inventory management, and distribution. It depends on **historical demand data** — knowing how much a product sold in previous months to predict the next.

But product codes change constantly: reformulations, repackaging, regulatory requirements, market restructuring. Each change breaks the demand history — the new code starts with **zero history**, even though it's the same product.

✗ WITHOUT CHAINS

Only the current code's history is visible. Years of data are lost.

`1003`

✓ WITH CHAINS

All predecessors are linked. Full history is preserved.

`1001` → `1002` → `1003`

Walkthrough Example

A shampoo product goes through two code changes over three years. Here's how the chain system tracks it:

1. **January 2021 — Introduction**
   Product code **1001** is introduced in Poland (PL) as an IPC code.
2. **July 2022 — First Code Change**
   The formula is updated. A new code **1002** is introduced, chained to 1001, and 1001 is discontinued. The system records the link.
3. **April 2024 — Second Code Change**
   Repackaged into a new size. Code **1003** replaces 1002. Same process: introduce, chain, discontinue.

The resulting **Product Family**:

Product Family **PL/IPC-0001** — all three codes share the same demand history.

Splits & Merges

Families can also **split** (one code becomes two) or **merge** (two codes become one), forming a directed acyclic graph (DAG):

Key Concepts

Generation
A single code's lifespan — from introduction to discontinuation. One code can have multiple non-overlapping generations.

Chain
A link between two codes: a successor replaces a predecessor. Creates edges in the product family graph.

Product Family
All generations connected through chains form a family (DAG). Forecasting aggregates demand across the entire family.

Resolution
Look up which family a code belongs to on a given date, or find all active codes in a family on a given date.

Event Transition Types

Each event contains one or more transitions. There are three types:

TYPE	WHAT IT DOES	REQUIRED FIELDS
INTRO	Introduces a new product code — makes it active	`introduction_code` , `date`
DISCONT	Discontinues an active code — sets its end date	`discontinuation_code` , `date`
chain	Links a successor to a predecessor — creates a family edge	`introduction_code` , `discontinuation_code` , `date`

Example Event Payload

```
{
  "iso_country_code": "PL",
  "transitions_write": [
    {
      "code_type_id": "IPC",
      "type": "INTRO",
      "date": "2023-06-01",
      "introduction_code": 1002
    },
    {
      "code_type_id": "IPC",
      "type": "chain",
      "date": "2023-06-01",
      "introduction_code": 1002,
      "discontinuation_code": 1001
    },
    {
      "code_type_id": "IPC",
      "type": "DISCONT",
      "date": "2023-06-01",
      "discontinuation_code": 1001
    }
  ]
}
```

Validation Rules

Your API must reject these invalid scenarios with **HTTP 400**:

#	INVALID SCENARIO	REASON
1	Double introduction of an active code	Overlapping generations
2	Overlapping generation (intro at earlier date)	Time intervals must not overlap
3	Double discontinuation	Code already discontinued
4	Chain with non-existing discontinuation code	Predecessor must exist
5	Chain with non-existing introduction code	Successor must exist
6	Discontinuation of never-introduced code	Nothing to discontinue
7	Chain missing `discontinuation_code`	Structurally invalid
8	Intro missing `introduction_code`	Structurally invalid
9	Discont missing `discontinuation_code`	Structurally invalid
10	Chain where intro == discont code	Cannot self-replace
11	Invalid country code	Reference data missing
12	Invalid code type	Reference data missing
13	Missing date	Structurally invalid

How Testing Works

The provided test script validates your API in three phases:

PHASE 0
Setup
POST /seed/setup/ seeds reference data (countries & code types) and returns HTTP 200.

PHASE 1
Event Ingestion
~150 events are posted. Valid events must return 201, invalid ones must return 400. Then recomputation is triggered.

PHASE 2
Family Queries
Resolve and reverse-resolve queries verify that families were computed correctly.

API Endpoints

Setup

METHOD	ENDPOINT	DESCRIPTION
POST	`/api/setup/`	Seed reference data, return 200

Events

METHOD	ENDPOINT	DESCRIPTION
GET	`/api/events/`	List all events
POST	`/api/events/`	Create event with transitions
GET	`/api/events/{id}/`	Get single event
DELETE	`/api/events/{id}/`	Delete event

Product Families

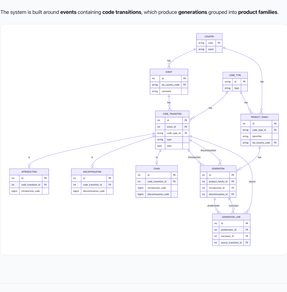
METHOD	ENDPOINT	DESCRIPTION
GET	`/api/product-families/`	List all families
GET	`/api/product-families/{id}/`	Get family with generations
POST	`/api/product-families/recompute/`	Trigger recomputation

Resolution

METHOD	ENDPOINT	DESCRIPTION
GET	`/api/resolve/`	Code + date + country → family
GET	`/api/resolve/reverse/`	Family + date → active codes
POST	`/api/resolve/bulk/`	Bulk resolve multiple codes

Data Model

The system is built around **events** containing **code transitions**, which produce **generations** grouped into **product families**.



Reference Data

COUNTRIES

PL	Poland
DE	Germany
US	United States
JP	Japan
CN	China

CODE TYPES

IPC	International Product Code
GTIN	Global Trade Item Number